Riesgo de default de tarjeta de crédito

Del experimento a producción con MLflow, Docker y Kubernetes

Luis Bryan Cueva Parada · Proyecto final de MLOps

El problema: predecir quién no pagará el próximo mes

- Dataset UCI *Default of Credit Card Clients*
- Clientes de un banco de Taiwán
- 23 variables: perfil, historial de pagos, facturas y pagos de 6 meses

30.000
clientes

22 %
cae en default: clases desbalanceadas

El dataset: qué sabemos de cada cliente

Grupo	Qué contiene	Ejemplo
Perfil	Cupo, edad, sexo, educación y estado civil	Cupo de 120.000, 35 años
Historial de pagos	Si estuvo al día o atrasado en cada uno de los últimos 6 meses	0 = al día, 3 = 3 meses de atraso
Facturas	Cuánto le facturaron cada mes	40.000
Pagos	Cuánto pagó cada mes	2.000
A predecir	**`default`: ¿dejó de pagar el mes siguiente?**	**1 = no pagó, 0 = pagó**

Cada fila es un cliente. Son 23 variables de entrada y una a predecir. Montos en dólares taiwaneses (NT$).

El ciclo: seguimos el camino del modelo

Datos → Entrenar → Registrar → Docker → Kubernetes → API + UI → Monitoreo

Si el drift dispara la alarma → reentrenar → comparar → mover el alias

Tres pilares

Escalable, auditable y fiable

Tres cosas versionadas

Código en Git, datos con hashes, modelo en MLflow

Datos: separación reproducible y auditable

Grupo	Filas	Uso	Default
Train	18.000	Entrenar	22,1 %
Test	6.000	Evaluar	22,1 %
Pool	6.000	Simular producción y drift	22,1 %

- Semilla 42 y separación estratificada
- `manifest.json` guarda los hashes, y cada run los registra en MLflow

Mostrar: `data/reference/manifest.json`

Experimentos: 8 runs comparados en MLflow

Mejor run de cada tipo	Runs	ROC-AUC
Random forest (300 árboles)	**3**	**0,7774**
Hist gradient boosting	3	0,7754
Regresión logística	2	0,7558

Mostrar: MLflow → `credit-default-training`, ordenado por `roc_auc`

¿Cómo sabemos si el modelo es bueno? ROC-AUC

Lo que necesita el banco

Saber **a quién vigilar primero**: ordenar a los clientes de mayor a menor riesgo.

Dónde poner el corte para aprobar o rechazar es una decisión del negocio.

Lo que mide ROC-AUC

Tomo un cliente que pagó y otro que no. ¿El modelo le da más riesgo al que no pagó?

77,7 %

de esas parejas las ordena bien

0,5 = azar · 1 = perfecto

¿Por qué no contar aciertos? Con 22 % de default, decir siempre «paga» acierta el 78 % sin detectar a nadie.

Registry: el alias decide qué modelo va a producción

Versión	Modelo	Alias
v1	Regresión logística	
v2	Hist gradient boosting	
v3	**Random forest**	**`@champion`**
v4	Reentrenado por drift	`@challenger`

La API pide `@champion`, nunca una ruta de archivo.

Mostrar: MLflow → Models, y `localhost:30080/info`

API y Docker: el modelo no va dentro de la imagen

API (FastAPI)

- `/predict`: la predicción
- `/info`: qué modelo sirve
- `/health`: ¿está vivo?
- `/ready`: ¿ya cargó el modelo?

Imagen Docker

- Solo código y dependencias
- Carga `@champion` al arrancar
- Cambiar de modelo no requiere reconstruir

Mostrar: `Dockerfile` y `localhost:30080/docs`

Kubernetes: declaro el estado y él lo mantiene

```
replicas: 3                # 3 copias de la API
image: credit-api:1.0      # la imagen
readinessProbe: /ready     # sin modelo, no recibe tráfico
livenessProbe: /health     # si no responde, se reinicia
memory: 768Mi              # requests = limits
```

- **Deployment:** mantiene las réplicas
- **Service:** dirección fija que reparte el tráfico

Mostrar: `k8s/20-api.yaml` y `kubectl get pods -n mlops`

Demostraciones: los cuatro requisitos, medidos

Prueba	Resultado	Script
Réplicas	3/3 pods Running	`demo_1`
Balanceo	30 peticiones entre los 3 pods	`demo_2`
Self-healing	Pod borrado y reemplazado: 157 OK, 0 fallidas	`demo_3`
Escalado	3 → 5 → 3 réplicas	`demo_4`

En vivo: `.\scripts\demo_3_self_healing.ps1`

Data drift: ¿cambiaron los clientes que llegan?

Qué es

El modelo aprendió con cierto tipo de cliente. Si ahora llegan, por ejemplo, clientes más jóvenes y con más cupo, ya no se parecen a los que conoce.

Cómo se detecta

Comparo cada variable contra los datos de entrenamiento: el cambio debe ser real (KS, Chi^2) y grande (PSI).

Resultado

Lote normal: **VERDE**, 0 variables

Lote alterado: **ROJO**, 15 variables

200 lotes sanos: 0 falsas alarmas

Acción: vigilar · **En vivo:** `.\scripts\demo_5_drift.ps1`

Reentrenamiento: challenger contra champion

Comparación en los lotes recientes, después del drift:

Modelo	ROC-AUC reciente
Champion v3	0,611
Challenger v4	**0,808**

Promover es una decisión humana: mover el alias y reiniciar los pods de forma gradual, sin reconstruir la imagen.

Comando: `kubectl rollout restart deployment/credit-api -n mlops`

Conclusión

El modelo no termina al entrenarlo: se registra, se despliega, se vigila y se reemplaza sin detener el servicio.

¿Preguntas?